

PROYECTO I

Clasificación de Landmarks mediante
Redes Neuronales

1. Contexto del Proyecto

Los servicios de almacenamiento y compartición de fotos necesitan datos de ubicación para funciones como sugerencia automática de etiquetas y organización de imágenes. Sin embargo, muchas fotos carecen de metadatos GPS, ya sea porque la cámara no tiene GPS o por razones de privacidad.

Una forma de inferir la ubicación es detectar y clasificar **landmarks** (puntos de referencia reconocibles) en la imagen. Dado el volumen masivo de imágenes, se requiere una solución automatizada basada en **Deep Learning**.

En este proyecto construirán modelos CNN para predecir automáticamente la ubicación de una imagen basándose en los landmarks que contenga. El dataset incluye imágenes de todo el mundo, divididos en conjuntos de entrenamiento y pruebas.

2. Objetivo General

Diseñar, entrenar, evaluar y desplegar modelos de clasificación de imágenes de landmarks utilizando PyTorch, aplicando tanto un modelo CNN propio como Transfer Learning, alcanzando los umbrales mínimos de rendimiento definidos en este documento.

3. Requerimientos Técnicos

3.1 Herramientas y Librerías Requeridas

- Python 3.7+ con PyTorch y torchvision
- Jupyter Notebook
- GPU recomendada (NVIDIA con CUDA)

3.2 Dataset

Se les proporcionará el dataset **Landmark Classification**, un subconjunto del Google Landmarks Dataset v2. El dataset viene dividido en carpetas **train/** y **test/**, donde cada subcarpeta corresponde a una clase.

4. Fases del Proyecto

Fase 1: Exploración y Preprocesamiento de Datos

1. Cargar el dataset utilizando **torchvision.datasets.ImageFolder** o una estructura personalizada.
2. Visualizar al menos 5 imágenes de ejemplo con sus etiquetas correspondientes.
3. Analizar la distribución de clases (gráfico de barras con cantidad de imágenes por landmark).
4. Implementar transformaciones de datos: redimensionado a 256px, recorte central a 224px, normalización con media y desviación estándar de ImageNet.
5. Aplicar **data augmentation** al conjunto de entrenamiento (rotaciones aleatorias, flip horizontal, cambios de color, etc.).
6. Crear los DataLoaders para train, validation (split del train) y test.

Validación: Los DataLoaders deben generar batches correctos

Fase 2: CNN desde Cero (From Scratch)

Tareas:

1. Diseñar una arquitectura CNN propia con al menos 3 capas convolucionales.
2. Definir la función de pérdida y el optimizador.
3. Entrenar el modelo por al menos 30 épocas, registrando la pérdida de entrenamiento y validación en cada época.
4. Graficar las curvas de pérdida (train vs. validation) y accuracy.
5. Guardar el mejor modelo (menor loss de validación).
6. Exportar el modelo usando **Torch Script (torch.jit)**.

Métrica	Umbral Mínimo
Test Accuracy	≥ 40%

Fase 3: CNN con Transfer Learning

1. Seleccionar dos modelos preentrenados de torchvision (ResNet18, ResNet50, VGG16, etc.).
2. Justificar por escrito la elección del modelo preentrenado.
3. Congelar las capas del feature extractor y reemplazar la capa clasificadora final.
4. Entrenar y evaluar el modelo con las mismas métricas que la Fase 2.
5. Graficar las curvas de entrenamiento (loss y accuracy).
6. Guardar y exportar el mejor modelo con Torch Script.
7. Comparar los resultados con los modelos de la Fase 2 en una tabla o gráfico.

Métrica	Umbral Mínimo
Test Accuracy	≥ 70%

Fase 4: Aplicación de Predicción

1. Crear una función **predict_landmarks(img_path, k)** que reciba la ruta de una imagen y devuelva los top-k landmarks más probables con sus probabilidades.
2. Utilizar el mejor modelo (Torch Script exportado) para realizar la inferencia.
3. Probar la función con al menos 4 imágenes propias (descargadas de internet o fotos personales) y mostrar los resultados visualmente.
4. Incluir un análisis escrito sobre las fortalezas y debilidades del modelo basado en las predicciones.

Validación: La función debe retornar resultados coherentes y las imágenes de prueba deben mostrarse junto a las predicciones.

Los motivamos a agregar una nueva categoría. Santa Cruz de la Sierra, Uyuni, etc y verificar que impacto tiene en el aprendizaje.

5. Entregables

- **Repositorio GitHub público** con todo el código fuente organizado. O una carpeta .zip cargada a la plataforma.
- **Notebook(s) Jupyter ejecutado(s)** (.ipynb) con todas las celdas corridas y salidas visibles.
- **README.md** con instrucciones de instalación, ejecución y descripción del proyecto.
- **NO incluir** el dataset de imágenes en el repositorio.
- Además debe incluir un video de 3 a 5 min explicando sus resultados y decisiones. Cargar el video a Youtube y compartir el enlace.

6. Estructura Sugerida del Repositorio

landmark-classifier/

```
├── README.md
├── cnn_from_scratch.ipynb
├── transfer_learning.ipynb
├── app.ipynb
├── src/
│   ├── data.py
│   ├── model.py
│   ├── train.py
│   └── predictor.py
├── models/
│   └── (modelos .pt exportados)
└── report.pdf
```

7. Rúbrica de Evaluación (20 puntos)

Cada criterio se evalúa según el cumplimiento de los requerimientos. El puntaje total es de 20 puntos.

Criterio	Descripción	Puntos
FASE 1: Exploración y Preprocesamiento (2 pts)		
Visualización del dataset	Se muestran imágenes de ejemplo con etiquetas y se incluye un gráfico de distribución de clases.	1
Transformaciones	Se implementan correctamente resize, crop, normalización y data augmentation para entrenamiento.	
DataLoaders	Se crean DataLoaders funcionales para train, validation y test con batch size adecuado.	1
FASE 2: CNN desde Cero (5 pts)		
Arquitectura CNN	Diseño propio con ≥ 3 capas conv, pooling, dropout y FC. Salida de 50 clases. Se justifica la arquitectura.	2
Entrenamiento	Se entrena ≥ 30 épocas, se registran métricas por época y se grafican curvas de loss y accuracy (train vs val).	1
Rendimiento	Test accuracy $\geq 40\%$. Se guarda el mejor modelo y se exporta con Torch Script.	2
FASE 3: Transfer Learning (7 pts)		
Selección del modelo	Se elige un modelo preentrenado, se justifica la elección por escrito y se adapta correctamente (congelar + nueva FC).	2
Entrenamiento	Se entrena, se grafican curvas de loss/accuracy y se compara con el modelo scratch en tabla o gráfico.	1
Rendimiento / Pruebas	Test accuracy $\geq 70\%$. Supera al modelo de la Fase 2. Se exporta con Torch Script. Se prueban ≥ 4 imágenes propias y se muestran resultados visuales junto a las predicciones.	2
Análisis	Se incluye reflexión escrita sobre fortalezas, debilidades del modelo y posibles mejoras.	2
FASE 4: Presentación (6 pts)		
Video y documentación	Se incluye reflexión escrita sobre fortalezas, debilidades del modelo y posibles mejoras. El video es coherente con el informe y está dentro del tiempo requerido de 3 a 5 min. Se expresa con claridad los conceptos y temas abordados.	6

TOTAL	20 pts
--------------	---------------

8. Criterios Adicionales

Penalizaciones

- Notebook no ejecutado (sin salidas visibles): **-3 puntos**
- Dataset incluido en el repositorio: **-1 punto**
- Plagio o copia directa de código sin comprensión demostrada: **0 puntos en el proyecto**

Bonificación (hasta +2 puntos extra)

- Implementar una interfaz interactiva (Gradio, Streamlit) para la predicción: **+1 punto**
- Superar 75% de accuracy en test con el modelo de Transfer Learning: **+1 punto**

9. Recomendaciones

- Utilicen **GPU** para el entrenamiento (Google Colab gratuito ofrece acceso a GPU).
- Guarden checkpoints del modelo durante el entrenamiento para no perder progreso.
- Experimenten con diferentes hiperparámetros: learning rate, batch size, número de épocas, optimizador (SGD, Adam).
- Para Transfer Learning, prueben primero con las capas congeladas y luego consideren descongelar algunas capas finales (fine-tuning) si el rendimiento no es suficiente.
- Documenten sus decisiones y razonamiento en celdas Markdown del notebook.
- Consulten la documentación oficial de PyTorch: **<https://pytorch.org/docs/>**